

Day 16 – OWASP Top 10 Vulnerabilities

Most Critical Risks Causing Real-World Data Breaches

1 A01 – Broken Access Control

What is it?

- Users can access data or functions **they are not allowed to**
- Example:

- Viewing another user's order
- Accessing admin pages as a normal user

Why it happens

- Missing server-side authorization checks
- Trusting URLs or client-side logic

Privilege Escalation via Parameter Tampering: E-commerce Example

Scenario Setup

Example:

- Site: "[ShopEasy.com](https://shopeasy.com)" - an online store
- Admin: Alex (User ID: 1, Role: "administrator")
- Attacker: Bob (User ID: 12345, Role: "customer")
- Vulnerability: Profile update API trusts client-sent role parameter

Step-by-Step Attack Visualization

Phase 1: Discovery (Bob the Customer)

Bob's Normal Account:

- Email: bob@gmail.com
- Role: "customer"
- Permissions: Can only buy products, view own orders
- Cannot: See other users' data, modify products, access admin panel

Bob visits his profile page:

text

URL: https://shopeasy.com/my-account

What Bob sees on screen:

My Profile

Email: [bob@gmail.com] ← Only field he can edit

[Save Changes]

But the ACTUAL HTML code contains:

html

```
<!-- Hidden fields user doesn't see but browser sends -->
<input type="hidden" name="user_id" value="12345">
<input type="hidden" name="user_role" value="customer">

<input type="text" name="email" value="bob@gmail.com">
```

Phase 2: Interception & Tampering

Bob opens Browser Developer Tools (F12):

Step 1: Bob clicks "Save Changes"

Step 2: Browser sends this normal request:

```
http
POST /api/update-profile HTTP/1.1
Host: shopeasy.com
Cookie: session=abc123
Content-Type: application/json

{
  "user_id": "12345",
  "email": "bob@gmail.com",
  "user_role": "customer" // ← Hidden field sent automatically
}
```

Step 3: Bob intercepts the request using:

1. Browser's Network tab → finds the POST request
2. Burp Suite proxy (professional tool)
3. TamperData extension (browser add-on)

Step 4: Bob modifies the request BEFORE it reaches server:

```
http
POST /api/update-profile HTTP/1.1
Host: shopeasy.com
Cookie: session=abc123
Content-Type: application/json

{
  "user_id": "12345",          // Still Bob's ID
  "email": "bob@gmail.com",
  "user_role": "administrator" // ← CHANGED FROM "customer"
}
```

Note : What just happened?

- Bob told the server: "Hey, I'm Bob (ID 12345) and I'm an administrator"
 - The server believes him because it doesn't verify if Bob is allowed to change his own role
-

Phase 3: The Vulnerable Server Code

What happens on ShopEasy's server:

```
python
# backend/api.py - VULNERABLE CODE
@route('/api/update-profile', methods=['POST'])
def update_profile():
    data = request.get_json() # Gets Bob's tampered data
    # FLAW 1: Uses user_id from CLIENT (should use session)
    user = User.query.get(data['user_id'])
    # FLAW 2: No permission check!
    # Should verify: "Is logged-in user allowed to change user 12345's role?"
    # Missing: if current_user.id != user.id: return error

    # FLAW 3: Directly sets role from client data
    user.role = data['user_role'] # Sets Bob to "administrator"
    user.email = data['email']

    db.session.commit()

    return {"message": "Profile updated successfully"}
```

The server should have:

```
python
# CORRECT CODE
def update_profile():
    current_user = get_user_from_session() # From session, NOT request
```

A01 – Broken Access Control

```
# Only allow updating YOUR OWN profile
if data['user_id'] != current_user.id:
    return error("Cannot update other users")

# NEVER let users change their own role
if 'user_role' in data:
    return error("Role cannot be changed")

# Only update allowed fields

current_user.email = sanitize(data['email'])
```

Phase 4: Consequences - Bob Becomes "Admin"

After the tampered request succeeds:

1. Database changes:

sql

```
-- Before attack:
SELECT * FROM users WHERE id = 12345;
-- id | email          | role
-- 12345| bob@gmail.com  | customer

-- After attack:
SELECT * FROM users WHERE id = 12345;
-- id | email          | role
-- 12345| bob@gmail.com  | administrator ← ROLE CHANGED!
```

2. Bob's new capabilities:

text

https://shopeasy.com/admin/dashboard	← Now ACCESSIBLE!
https://shopeasy.com/admin/products	← Can edit/delete products
https://shopeasy.com/admin/orders	← View ALL customer orders
https://shopeasy.com/admin/users	← View/modify ALL user accounts

Phase 5: Real Damage Examples

Attack 1: Data Theft

Bob accesses: /admin/export-users

Downloads: 50,000 customer records including:

- Names, emails, phone numbers
- Home addresses
- Purchase history
- Credit card last-4 digits

Sells on dark web for \$5,000

Attack 2: Financial Fraud

Bob creates "refund" for himself:

Product: iPhone 14 (never purchased)

Amount: \$999

To: Bob's PayPal

Total stolen: \$50,000 across 50 fake refunds

Attack 3: Business Sabotage

Bob changes all prices:

iPhone 14 → \$0.99

MacBook Pro → \$1.50

Customers place 1,000 orders before detected

Company loses \$2,000,000

Why This Happens - The Root Cause

Developers made these assumptions:

1. "Users won't see hidden fields" → Wrong: Browser tools show everything
2. "Users won't modify requests" → Wrong: Easy with proxies
3. "Client data can be trusted" → WRONG: Never trust the client!

A01 – Broken Access Control

The Golden Rule:

"Never let the client tell you what their permissions should be. Decide permissions SERVER-SIDE based on authenticated session."

Simple Fix - Server-Side Validation

```
def update_profile(request):  
    # 1. Get current user FROM SESSION (not from request data)  
    current_user = get_current_user_from_session_token()  
  
    # 2. Only allow updating current user's own profile  
    requested_user_id = request.data.get('user_id')  
    if requested_user_id != current_user.id:  
        return error("Can't update other users")  
  
    # 3. NEVER accept role from client  
    if 'user_role' in request.data:  
        # Silently ignore or return error  
        del request.data['user_role']  
  
    # 4. Only update whitelisted fields  
    allowed_fields = ['email', 'phone', 'address']  
    for field in request.data:  
        if field not in allowed_fields:  
            return error(f"Cannot update {field}")  
  
    # 5. Update safely  
  
    update_user(current_user.id, request.data)
```

Result: Bob can only change his email/phone, never his role. The server decides roles, not the client.

This is vertical privilege escalation - moving from "customer" to "administrator" by tampering with parameters the server shouldn't have trusted.